Optimizing $500M of NYC school funding

A linear program that decides which of 349 Title I high schools get discretionary money — and proves its own numbers.

Mukunth Gopalakrishnan · ORIE 4820 capstone, rebuilt as production software

The problem

NYC DOE allocates discretionary Title I funding on financial need alone

Poverty is the only input

Schools are ranked by the share of students on free or reduced-price lunch. Academic performance never enters the decision.

But money barely tracks outcomes

Per-pupil funding and Regents scores correlate at r = −0.16 — about 2% of the variance, and negative. The best-funded schools score lowest.

So the gap persists

Schools that are both poor and academically behind are not prioritised over schools that are only poor.

Source: NYC DOE school allocation and Regents reporting, 349 Title I high schools, 210,396 students.

What I built

A spreadsheet and two ad-hoc scripts, rebuilt as a tested package

1	Need model	Blends financial and academic need into one score per school
2	Optimizer	Linear program over 349 schools with per-pupil caps and a guaranteed floor
3	Run registry	Every run stores the exact parameters that produced it
4	Comparison engine	Overlays scenarios and ranks them on a fixed yardstick
5	Interfaces	CLI, Streamlit dashboard, standalone HTML report, questionnaire

11 CLI commands · 4,747 lines · 239 tests

How the allocation works

Score need, then solve for the best placement of a fixed budget

1 · Score need

Two z-scores per school: the share on free lunch, and the Regents average inverted so low scores mean high need.

$$\text{weight} = 0.30 \cdot z(\text{FRPL}) + 0.70 \cdot (-z(\text{score}))$$

Schools above the floor form the high-need tier: 171 of 349.

2 · Allocate

$$\text{maximise} \quad \Sigma\, w_i \cdot x_i$$

$$\text{subject to} \quad \Sigma\, x_i \le \$500\text{M}$$

$$l_i \le x_i \le \text{cap}_i \cdot \text{enrollment}_i$$

Caps of $5,000 per pupil for high-need schools and $1,500 for the rest turn “maximise weighted need” into a priority queue. Schools below 60 points are guaranteed a floor award.

Solved with scipy.optimize.linprog (HiGHS). Outcomes are projected under a stated linear impact assumption — documented as an assumption, not an estimate.

Results at $500M

The whole budget placed; 335 of 349 schools funded

+5.00

Regents points gained by the average high-need school

55%

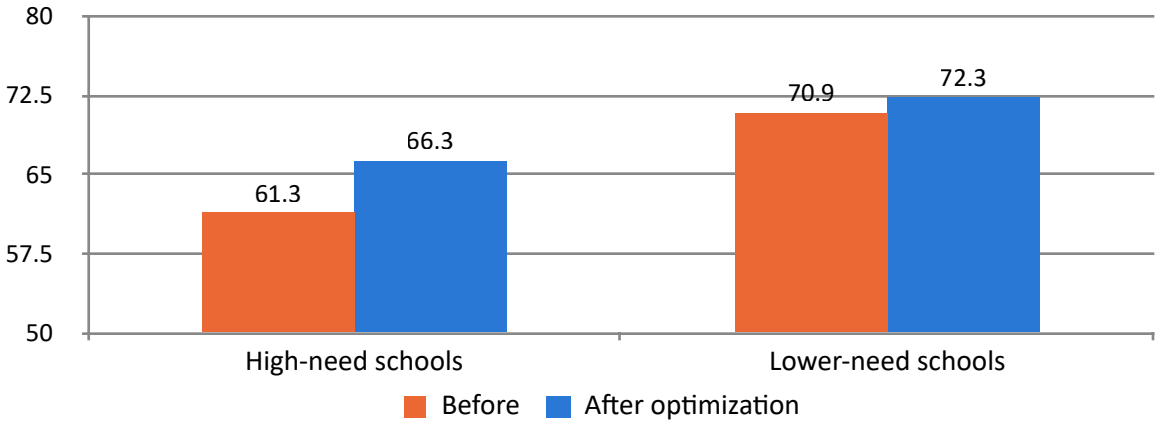
of the budget to schools serving 26% of the students

3.4×

better targeting than the status quo, per dollar spent

0.0

difference from the published analysis it reproduces



The trade-off, stated plainly

High-need schools take 55% of the money while serving 26% of the students.

The Gini coefficient of per-pupil funding rises from 0.099 to 0.110 — funding becomes less equal, on purpose.

That is the model working as designed, and a value judgement a reader can reject.

What the single-scenario analysis missed

Two ways this model misleads you — both found by building the comparison tool

Return per dollar is a mirage

A school's gain is funding ÷ enrollment × k, so its student-points are funding × k. Summed, that is budget × k.

Total return depends only on how much you spend — never on where. My first ROI metric read exactly 1,000.000 for all five scenarios.

So “which allocation is better” is never an efficiency question here. Only targeting differs.

The yardstick moved

“Need-tier gap closed” sounded like the right metric — but each scenario defines its own tiers from its own weights.

Ranked that way, the status quo appeared to beat the proposed model. The ordering was an artifact of the groups moving.

Fixed by measuring between baseline score quartiles, which no scenario can shift.

Both are enforced in code and printed as warnings, so the next person cannot make the mistake I nearly shipped.

Built to be trusted

The analysis is auditable, and the tooling proves it on every commit

Golden regression

Reproduces every published per-school allocation at max difference 0.0 — checked on each commit.

Run provenance

Each run stores its full parameter set and a content hash, so results are never anonymous.

titlei verify

Nine checks from Python version to reproducing the published numbers. For users who will never run pytest.

CI across 3.9–3.12

Lint, 239 tests at 96% coverage, the guided demo and the full pipeline, on four interpreters.

Seven latent bugs were found by running the documented commands verbatim — twice. Idempotency failures only appear on the second run.

What this project shows

- **Modelling** — Constrained optimisation applied to a real policy question, with the assumptions written down rather than hidden.
- **Engineering** — A frozen spreadsheet turned into a tested, documented, reproducible package anyone can run in three commands.
- **Judgment** — Found and corrected two metrics that would have told a flattering but wrong story.

Python · SciPy (HiGHS) · pandas · matplotlib · Streamlit · pytest · GitHub Actions